

Competitive Programming

Section 25 Editorial: Interval DP

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	4
2	Tool reminder	5
3	A. Merge Piles	6
3.1	Restatement	6
3.2	Constraints and traps	6
3.3	Observations	6
3.4	Sample walkthrough	6
3.5	Algorithm	7
3.6	Correctness proof	7
3.7	Complexity	7
3.8	Full C++ solution	7
3.9	Full Python solution	8
3.10	Common wrong answers	9
3.11	Tests that matter	9
4	B. Palindrome Repairs	10
4.1	Restatement	10
4.2	Constraints and traps	10
4.3	Observations	10
4.4	Sample walkthrough	10
4.5	Algorithm	10
4.6	Correctness proof	11
4.7	Complexity	11
4.8	Full C++ solution	11
4.9	Full Python solution	12
4.10	Common wrong answers	12
4.11	Tests that matter	12
5	C. Treasure Balloons	14
5.1	Restatement	14
5.2	Constraints and traps	14
5.3	Observations	14
5.4	Sample walkthrough	14
5.5	Algorithm	15
5.6	Correctness proof	15
5.7	Complexity	15
5.8	Full C++ solution	15
5.9	Full Python solution	16
5.10	Common wrong answers	17
5.11	Tests that matter	17
6	Testing and stress strategy	18

1 Editorial overview

Problem	Interval meaning	Transition type
A. Merge Piles	minimum merge cost for piles $l..r$	final split
B. Palindrome Repairs	minimum insertions for substring $l..r$	endpoints
C. Treasure Balloons	maximum score for popping $l..r$	last removed item

All problems are solved by increasing interval length.

2 Tool reminder

For interval DP:

```
for length from small to large:  
    compute every interval of that length
```

This guarantees dependencies such as $[l+1, r]$, $[l, r-1]$, $[l, k]$, and $[k+1, r]$ are already known.

3 A. Merge Piles

3.1 Restatement

Piles are arranged in a row. One move merges two adjacent groups and costs the total number of stones in the resulting group. Find the minimum total cost to merge all piles into one group.

3.2 Constraints and traps

$n \leq 200$, so $O(n^3)$ is acceptable. Costs can exceed 32-bit integers.

Important traps:

```
Only adjacent groups can be merged.  
The final merge of interval [l,r] must split it into two adjacent intervals.  
The cost of the final merge is sum(l..r).  
Use prefix sums for interval totals.
```

3.3 Observations

If the final merge for interval $[l,r]$ combines:

```
[l,k] and [k+1,r]
```

then those two subintervals must already have been fully merged. Their optimal costs are independent once the final split is fixed.

3.4 Sample walkthrough

For:

```
10 20 30 40
```

One optimal sequence:

```
merge 10+20 -> 30, cost 30  
merge 30+30 -> 60, cost 60  
merge 60+40 -> 100, cost 100
```

Total: 190.

3.5 Algorithm

```
build prefix sums
dp[i][i] = 0

for length from 2 to n:
    for l:
        r = l + length - 1
        dp[l][r] = infinity
        for k from l to r-1:
            dp[l][r] = min(dp[l][r],
                           dp[l][k] + dp[k+1][r] + sum(l..r))

print dp[0][n-1]
```

3.6 Correctness proof

Lemma. In any complete merge of interval $[l, r]$, the final operation combines two fully merged adjacent intervals $[l, k]$ and $[k+1, r]$ for some k .

Proof. Immediately before the final merge, all piles in $[l, r]$ have been merged into exactly two adjacent groups. Since merges preserve order and only combine adjacent groups, those groups must cover $[l, k]$ and $[k+1, r]$. $\square$

Theorem. $dp[l][r]$ equals the minimum cost to merge interval $[l, r]$.

Proof. Induct on interval length. Length-one intervals need no merge, so cost 0 is correct. For longer intervals, the lemma says every valid strategy has some final split k . By induction, the best way to prepare the left and right groups costs $dp[l][k]$ and $dp[k+1][r]$; the final merge adds $\text{sum}(l..r)$. The recurrence tries every possible final split and takes the minimum, so it finds exactly the optimum. $\square$

3.7 Complexity

```
Time:  $O(n^3)$ 
Memory:  $O(n^2)$ 
```

3.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```

int n;
if (!(cin >> n)) return 0;
vector<long long> a(n), pref(n + 1, 0);
for (int i = 0; i < n; i++) {
    cin >> a[i];
    pref[i + 1] = pref[i] + a[i];
}
auto total = [&](int l, int r) {
    return pref[r + 1] - pref[l];
};
const long long INF = (1LL << 62);
vector<vector<long long>> dp(n, vector<long long>(n, 0));
for (int len = 2; len <= n; len++) {
    for (int l = 0; l + len <= n; l++) {
        int r = l + len - 1;
        long long best = INF;
        long long add = total(l, r);
        for (int k = l; k < r; k++) {
            best = min(best, dp[l][k] + dp[k + 1][r] + add);
        }
        dp[l][r] = best;
    }
}
cout << dp[0][n - 1] << '\n';
return 0;
}

```

3.9 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    a = data[1:1 + n]
    pref = [0]
    for x in a:
        pref.append(pref[-1] + x)

    def total(l: int, r: int) -> int:
        return pref[r + 1] - pref[l]

    inf = 10**30
    dp = [[0] * n for _ in range(n)]
    for length in range(2, n + 1):

```

```

    for l in range(0, n - length + 1):
        r = l + length - 1
        best = inf
        add = total(l, r)
        for k in range(l, r):
            cand = dp[l][k] + dp[k + 1][r] + add
            if cand < best:
                best = cand
        dp[l][r] = best
    print(dp[0][n - 1])

if __name__ == "__main__":
    main()

```

3.10 Common wrong answers

Allowing non-adjacent piles to merge.
 Forgetting the final merge cost.
 Using $O(n)$ interval sums inside the split loop.
 Using int in C++.

3.11 Tests that matter

One pile -> 0.
 Three equal piles -> 5 for 1 1 1.
 Large pile values -> long long.
 Cases where greedy smallest adjacent merge is not enough.

4 B. Palindrome Repairs

4.1 Restatement

Given a string, insert the fewest characters needed to make it a palindrome.

4.2 Constraints and traps

$|s| \leq 1000$, so $O(n^2)$ is intended.

Important traps:

```
Only insertions are allowed, but any letter may be inserted anywhere.  
Matching endpoints do not need extra insertions.  
Single characters are already palindromes.
```

4.3 Observations

For substring $s[l..r]$:

If $s[l] == s[r]$, the endpoints can match each other, so solve the inside.

If $s[l] != s[r]$, one insertion is needed to match one endpoint:

```
insert s[l] near the right side -> solve [l+1,r]  
insert s[r] near the left side  -> solve [l,r-1]
```

4.4 Sample walkthrough

For:

```
abcda
```

The outer `a` characters match, so the problem reduces to `bcd`. That substring needs two insertions. Answer: 2.

4.5 Algorithm

```

dp[i][i] = 0

for length from 2 to n:
    for l:
        r = l + length - 1
        if s[l] == s[r]:
            dp[l][r] = dp[l+1][r-1]
        else:
            dp[l][r] = 1 + min(dp[l+1][r], dp[l][r-1])

print dp[0][n-1]

```

4.6 Correctness proof

Theorem. $dp[l][r]$ is the minimum insertions needed to make $s[l..r]$ a palindrome.

Proof. Induct on interval length. Length 1 substrings are already palindromes. For longer substrings, if endpoints match, there is no reason to insert a character to match either endpoint; the inside determines the answer. If endpoints differ, a final palindrome must match $s[l]$ with an inserted $s[l]$ on the right side, or match $s[r]$ with an inserted $s[r]$ on the left side. These cases cost one insertion plus the optimal repair of the remaining substring. The recurrence takes the better of the two exhaustive choices. $\square$

4.7 Complexity

Time: $O(n^2)$
Memory: $O(n^2)$

4.8 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) return 0;
    int n = (int)s.size();
    vector<vector<int>> dp(n, vector<int>(n, 0));
    for (int len = 2; len <= n; len++) {
        for (int l = 0; l + len <= n; l++) {
            int r = l + len - 1;

```

```

        if (s[l] == s[r]) {
            dp[l][r] = (l + 1 <= r - 1 ? dp[l + 1][r - 1] : 0);
        } else {
            dp[l][r] = 1 + min(dp[l + 1][r], dp[l][r - 1]);
        }
    }
}
cout << dp[0][n - 1] << '\n';
return 0;
}

```

4.9 Full Python solution

```

import sys

def main() -> None:
    s = sys.stdin.readline().strip()
    if not s:
        return
    n = len(s)
    dp = [[0] * n for _ in range(n)]
    for length in range(2, n + 1):
        for l in range(n - length + 1):
            r = l + length - 1
            if s[l] == s[r]:
                dp[l][r] = dp[l + 1][r - 1] if l + 1 <= r - 1 else 0
            else:
                dp[l][r] = 1 + min(dp[l + 1][r], dp[l][r - 1])
    print(dp[0][n - 1])

if __name__ == "__main__":
    main()

```

4.10 Common wrong answers

Treating this as edit distance with deletions and substitutions.
 Forgetting the length-two case when endpoints match.
 Filling intervals in the wrong order.

4.11 Tests that matter

Single character.
Already palindrome.
All distinct characters.
Even and odd lengths.

5 C. Treasure Balloons

5.1 Restatement

Pop all balloons in some order. Popping balloon i scores the product of its value and the nearest still-unpopped values on both sides, using 1 outside the array. Maximize total score.

5.2 Constraints and traps

$n \leq 200$, so $O(n^3)$ is acceptable.

Important traps:

```
The neighbors of a balloon change over time.  
Choosing the first popped balloon makes the subproblems hard.  
Choosing the last popped balloon fixes the outside neighbors.  
Use long long.
```

5.3 Observations

Add sentinel balloons:

```
a[0] = a[n+1] = 1
```

Let:

```
dp[l][r] = maximum score from popping all balloons l..r,  
           while balloons l-1 and r+1 are still present
```

If k is the last balloon popped in $[l, r]$, then its score is:

```
a[l-1] * a[k] * a[r+1]
```

because all other balloons inside $[l, r]$ are gone.

5.4 Sample walkthrough

For:

```
3 1 5 8
```

The optimal score is 167. The important modeling point is not the exact order; it is that every interval tries each possible last balloon.

5.5 Algorithm

```
add sentinels

for length from 1 to n:
    for l:
        r = l + length - 1
        dp[l][r] = max over k in [l,r]:
            dp[l][k-1] + dp[k+1][r] + a[l-1] * a[k] * a[r+1]

print dp[1][n]
```

5.6 Correctness proof

Lemma. For any strategy that pops all balloons in $[l,r]$, there is a unique balloon k that is popped last in that interval, and its scoring neighbors are $l-1$ and $r+1$.

Proof. A finite popping order has one last element. Before that final pop, all other balloons in $[l,r]$ are gone, while the interval definition keeps outside balloons $l-1$ and $r+1$ unpopped. Therefore those are the nearest neighbors. $\square$

Theorem. $dp[l][r]$ is the maximum score obtainable from interval $[l,r]$ under the outside-neighbor assumption.

Proof. Induct on interval length. Empty intervals have score 0. For a nonempty interval, the lemma says every valid strategy has some last balloon k . The pops before k split into independent left and right intervals, whose optimal scores are correct by induction. The final score for k is fixed by the outside neighbors. Trying every k and taking the maximum gives the optimal strategy. $\square$

5.7 Complexity

```
Time:  $O(n^3)$ 
Memory:  $O(n^2)$ 
```

5.8 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    vector<long long> a(n + 2, 1);
    for (int i = 1; i <= n; i++) cin >> a[i];
    vector<vector<long long>> dp(n + 2, vector<long long>(n + 2, 0));
    for (int len = 1; len <= n; len++) {
        for (int l = 1; l + len - 1 <= n; l++) {
            int r = l + len - 1;
            long long best = 0;
            for (int k = l; k <= r; k++) {
                best = max(best, dp[l][k - 1] + dp[k + 1][r] + a[l - 1] * a[k] * a[r + 1]);
            }
            dp[l][r] = best;
        }
    }
    cout << dp[1][n] << '\n';
    return 0;
}

```

5.9 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    a = [1] + data[1:1 + n] + [1]
    dp = [[0] * (n + 2) for _ in range(n + 2)]
    for length in range(1, n + 1):
        for l in range(1, n - length + 2):
            r = l + length - 1
            best = 0
            for k in range(l, r + 1):
                cand = dp[l][k - 1] + dp[k + 1][r] + a[l - 1] * a[k] * a[r + 1]
                if cand > best:
                    best = cand
            dp[l][r] = best
    print(dp[1][n])

```

```
if __name__ == "__main__":  
    main()
```

5.10 Common wrong answers

Choosing the first popped balloon and using changing neighbors incorrectly.
Forgetting sentinel values.
Using 0-based interval formulas with 1-based sentinels inconsistently.
Using int for products and totals.

5.11 Tests that matter

One balloon -> its value.
All values 1 -> answer n.
The classic 3 1 5 8 -> 167.
Large values -> long long.

6 Testing and stress strategy

Run:

```
CP_TARGET=solution python3 sections/25_interval_dp/check.py
```

Random tests cover:

Merge Piles:

small random arrays and one-pile cases.

Palindrome Repairs:

random strings over a small alphabet plus existing palindromes.

Treasure Balloons:

random values and sentinel boundary behavior.

For tiny brute force:

A: recursively try every adjacent merge order for $n \leq 8$.

B: compare with n - longest palindromic subsequence for small strings.

C: recursively try every next popped balloon for $n \leq 8$.

7 What to carry forward

Interval DP is about asking the right structural question:

What is the interval?

What is the final operation, endpoint choice, or split point?

Are all resulting subproblems shorter intervals?

When those answers are clear, the implementation is usually just increasing length order.